

CodeCache

A Project Report

Submitted by

**Arav Sharma
Anugrah Shetty
Ayush Garg**

Under the Guidance of

Prof. Vijayetha Thoday/Prof. Rojal Tuscano

for the course “Web Programming”

in partial fulfillment for the award of the degree of

BTECH /MBATECH

At



**Department of Computer Engineering,
MPSTME, NMIMS.**

APRIL,2025

DECLARATION

We, Arav Sharma, Anugrah Shetty, Ayush Garg, Roll No. B115, B120, B143 BTECH/MBATECH, IV semester understand that plagiarism is defined as anyone or combination of the following:

1. Un-credited verbatim copying of individual sentences, paragraphs or illustration (such as graphs, diagrams, etc.) from any source, published or unpublished, including the internet.
2. Un-credited improper paraphrasing of pages paragraphs (changing a few words phrases, or rearranging the original sentence order)
3. Credited verbatim copying of a major portion of a paper (or thesis chapter) without clear delineation of who did wrote what. (Source: IEEE, The institute, Dec. 2004)
4. We have made sure that all the ideas, expressions, graphs, diagrams, etc., that are not a result of my work, are properly credited. Long phrases or sentences that had to be used verbatim from published literature have been clearly identified using quotation marks.
5. We affirm that no portion of my work can be considered as plagiarism and I take full responsibility if such a complaint occurs. I understand fully well that the guide of the seminar/ project report may not be in a position to check for the possibility of such incidences of plagiarism in this body of work.

Names: Arav Sharma, Anugrah Shetty, Ayush Garg

Roll Nos. : B115, B120, B143

Place: Mumbai

Date: 08-04-2025

CERTIFICATE

This is to certify that the project entitled “CodeCache” is the bonafide work carried out by Arav Sharma, Anugrah Shetty, Ayush Garg of BTECH/MBATECH, MPSTME (NMIMS), Mumbai, during the IV semester of the academic year 2024-2025 , in partial fulfillment of the requirements for the Course Web Programming

Prof. Vijayetha Thoday/Prof. Rojal Tuscano

Internal Mentor

Examiner 1

Examiner 2

Table of contents

Contents

Introduction	1
Scope	1
Purpose	2
Description of Modules	3
Frontend Libraries & Tools.....	3
Backend (Node + Python Hybrid).....	4
Software, APIs and Database Used.....	4
APIs.....	4
Database	5
Softwares.....	6
Methods Implemented.....	7
Code Snippets.....	8
Conclusion and Future Scope.....	12
Applications	12

Introduction

Code Cache is a web-based application developed as a centralized solution for managing reusable code snippets. Designed for developers and students, the platform allows users to securely store, categorize, and access code segments written in various programming languages. The goal of this project was to build a functional and responsive web application that solves a common developer problem—losing track of useful code written across multiple projects or environments.

The application was built using React for the frontend due to its component-based structure and dynamic rendering capabilities. Supabase was used for both user authentication and real-time database functionality, enabling users to securely log in and manage their own snippet collections. The project is deployed on Vercel, providing seamless hosting and continuous deployment through GitHub integration.

The idea behind Code Cache is not just storage, but organization. Users can group their code snippets by category, language, or purpose, and perform full CRUD (Create, Read, Update, Delete) operations on them.

Scope

The scope of this project covers the end-to-end development and deployment of *Code Cache*, a web-based code snippet library designed to help developers save, organize, and manage reusable code across different programming languages. The project was developed using React and Supabase, with deployment handled via Vercel. The scope includes both technical implementation and functional coverage, ensuring the application is fully responsive, scalable, and user-friendly.

- **Frontend Development** using React, Tailwind CSS and styled-components to create a responsive and intuitive user interface for code snippet management.
- **User Authentication Integration** through Supabase Authentication, enabling secure login and access control for personalized snippet storage.
- **Real-Time Database Functionality** using Supabase Firestore to store, retrieve, update, and delete user-specific snippets instantly.
- **CRUD Operations for Snippets**, allowing users to create, read, update, and delete their code snippets with ease.

- **Categorization and Tagging**, enabling users to organize snippets by language or topic for faster access and better management.
- **Responsive Design** to ensure the application performs well on both desktop and mobile devices.
- **Deployment and Hosting** on Vercel, with continuous deployment setup linked to the GitHub repository for real-time updates.

Purpose

The main purpose of Code Cache Flame is to help developers save time and improve productivity by providing a convenient platform to store and organize frequently used code snippets. It addresses the common problem of losing track of useful code by offering a searchable and categorized snippet repository. The project aims to encourage good coding practices, knowledge sharing, and efficient code reuse.

Code snippets are important because they streamline development by allowing developers to reuse pre-written code blocks, saving time and effort, promoting consistency, and improving overall productivity.

- **Increased Productivity:**

Code snippets allow developers to avoid repetitive typing and searching for common code patterns, leading to faster development cycles.

- **Consistency and Standardization:**

Using snippets ensures that all team members use the same code blocks for specific tasks, promoting consistency and reducing potential errors.

- **Faster Learning:**

Snippets can be a great way to learn new programming languages or techniques by providing ready-made examples of common tasks.

- **Collaboration:**

Snippets can facilitate collaboration by allowing team members to easily share and reuse code, promoting a shared understanding of coding practices.

Description of Modules

Frontend Libraries & Tools

◆ Core Libraries:

- `react`
- `react-dom`

◆ Routing:

- `react-router-dom` – For navigation and state management between pages

◆ Build & Tooling:

- `vite` – Fast build tool and dev server
- `typescript` – Static typing
 - With type packages:
 - `@types/node`
 - `@types/react`
 - `@types/react-dom`
- `eslint` – Linting and code quality enforcement

◆ Styling:

- `tailwindcss` – Utility-first CSS framework
 - Includes:
 - `autoprefixer` – For CSS vendor prefixes
 - `postcss` – CSS processing tool

Frontend (React + Vite) Modules

- `react`, `react-dom` – Core React libraries
- `react-router-dom` – For routing between pages
- `styled-components` – For component-level CSS styling
- `axios` – For making HTTP requests to backend APIs
- `uuid` – For generating unique identifiers
- `vite` – Frontend build tool

Backend (Node + Python Hybrid)

- **Node.js Modules:**
 - `express` – Web server for handling API routes
 - `dotenv` – To load environment variables
 - `cors` – Enable cross-origin requests
 - `body-parser` – Parse incoming request bodies
- **Python Modules (for search + embedding logic):**
 - `openai` or `google-generativeai` – For AI interaction (Gemini/PaLM 2.0)
 - `sklearn` – Likely used for vector similarity (e.g., cosine similarity)
 - `numpy` – Numerical operations for vector handling
 - `pandas` – (If used) for structured data manipulation
 - `flask` (if any Python server components exist — TBD)
 - `supabase` Python SDK (if Supabase is accessed via Python)

Software, APIs and Database Used

APIs

1. Supabase Authentication API

- Handles user registration, login, logout, and session management.
- Functions like `signUp()`, `signInWithPassword()`, and `signOut()` are accessed via the Supabase JS client.
- Ensures secure authentication through JSON Web Tokens (JWT) and cookie-based sessions.

Supabase Database API (PostgREST)

- Provides RESTful API access to your PostgreSQL database.
- Enables operations like:
 - **Create:** Inserting new code snippets into the database.
 - **Read:** Fetching snippets based on user ID or categories.
 - **Update:** Modifying existing snippets.
 - **Delete:** Removing snippets from the database.

- All these operations are securely scoped to authenticated users.

2. Google AI API – Model – 2.0 Flash

Google Cloud Natural Language API to analyze and generate metadata for code snippets. This integration allows the platform to automatically create descriptive tags and summaries based on the content of the code provided by users.

Key Features of the Integration:

- **Automated Tag Generation:** By processing the code snippets through the Natural Language API, the system extracts relevant keywords and concepts, facilitating efficient categorization and retrieval.
- **Dynamic Bio Creation:** The API's analysis enables the generation of concise descriptions or "bios" for each snippet, providing users with immediate context and understanding of the code's functionality.
- **Enhanced Searchability:** With accurate tags and summaries, users can quickly locate and utilize code snippets, improving overall workflow efficiency.

Database

For *Code Cache*, we used **Supabase** as the backend platform, which provides a powerful combination of authentication, real-time database capabilities, and RESTful APIs — all built on top of **PostgreSQL**, a robust open-source relational database.

Supabase is known as the “open-source Firebase alternative,” but it gives full SQL control, excellent developer tooling, and easy integration with frontend frameworks like React. It was chosen for this project due to:

- Its ease of integration with React via the `@supabase/supabase-js` client.
- Built-in **authentication** and **real-time database** features.
- Ability to write SQL queries directly if needed, unlike NoSQL.

Supabase Integrations in the Project

1. Supabase Authentication

- Handles **secure user login and registration** using email/password.
- Stores and manages user sessions with JWT tokens.
- Ensures that all code snippets are associated with a unique authenticated user ID (`user_id`).

2. Supabase PostgreSQL Database

- Stores user data, code snippets, generated tags, bios, and prompts.
- Each snippet record includes fields like:
 - `title`

- `code`
 - `language`
 - `tags` (generated via Google API)
 - `bio` (generated summary)
 - `prompt` (prompt cache input)
 - `user_id` (to map to the authenticated user)
 - Data is structured in normalized tables, supporting efficient querying and filtering.
3. **Supabase Realtime & API Layer**
 - Data fetching, insertion, and updates are handled using Supabase's **auto-generated RESTful API**.
 - The app doesn't need a custom backend — everything is done via the Supabase JS SDK.
 - Real-time capabilities could be leveraged for future features like live collaboration.
 4. **Row-Level Security (RLS)**
 - Supabase uses **RLS policies** to ensure that users can only access their own data.
 - Secure by design, preventing unauthorized access at the database level.

Softwares

• **React.js**

The core frontend framework used to build the user interface. React's component-based architecture enabled a modular, maintainable structure, perfect for building dynamic views like the dashboard, snippet creation, and browsing features.

• **Supabase**

A full backend-as-a-service platform used for:

- **Authentication** (user login/signup)
- **PostgreSQL database** (to store code snippets, user data, prompt history)
- **API layer** for handling real-time interactions between the frontend and backend.

• **Styled-Components**

A CSS-in-JS library used for styling the app. It allows you to write scoped CSS directly within React components, improving reusability and organization.

• **React Router DOM**

Used to implement client-side routing. Enables navigation between login, register, and dashboard pages without full-page reloads.

• **Google Cloud Natural Language API**

Integrated to analyze code snippets and generate meaningful **tags** and **bios** based on the content. This API adds an intelligent layer that enhances snippet discoverability and organization.

• **UUID (Universally Unique Identifier)**

Used to generate unique IDs for snippets or users where needed. Ensures data uniqueness across entries.

- **Dotenv**

Used to manage environment variables securely (such as API keys for Supabase and Google Cloud), keeping sensitive information out of the codebase.

- **Vercel**

The platform used for deploying the live version of the site. It supports auto-deployment from GitHub on every commit, ensuring continuous delivery.

- **Git & GitHub**

Git was used for version control, while GitHub was the central repo for code collaboration, issue tracking, and CI/CD integration.

Python (Custom Search Logic)

A Python script/module was developed to handle:

- **Vector-based search:** Code snippets and bios are converted into embeddings or tokenized data structures for similarity comparison.
- **Word-level matching:** Searches are performed not just by exact strings but by matching relevant terms found in bios, allowing more intelligent, fuzzy search results.
- This module works alongside the database and API layer to return relevant results based on semantic and keyword matches.

Methods Implemented

- **User Authentication**

- Implemented via **Supabase Auth**, allowing users to register, log in, and manage their data securely using email/password credentials.

- **Code Snippet CRUD Operations**

- Users can **Create, Read, Update, and Delete** code snippets.
- Each snippet includes a title, language, raw code, and is enriched with auto-generated tags and a bio.

- **Prompt Cache System**

- A unique feature where the **input prompt sent to the Google API** is also stored alongside the generated output.
- This lets users revisit not only the snippet and the AI's result (tags/bio), but also understand *how* it was generated, making the process transparent and traceable.

- **Google Cloud Natural Language API Integration**

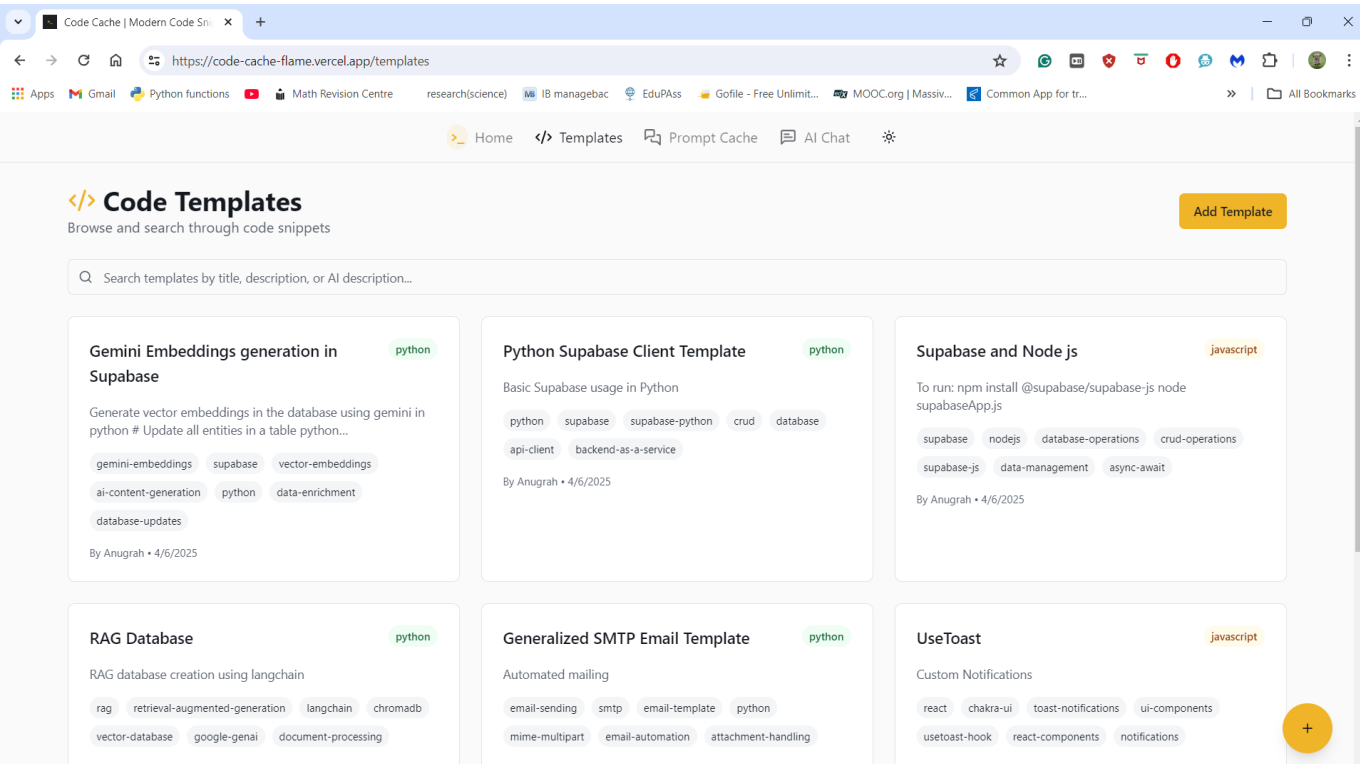
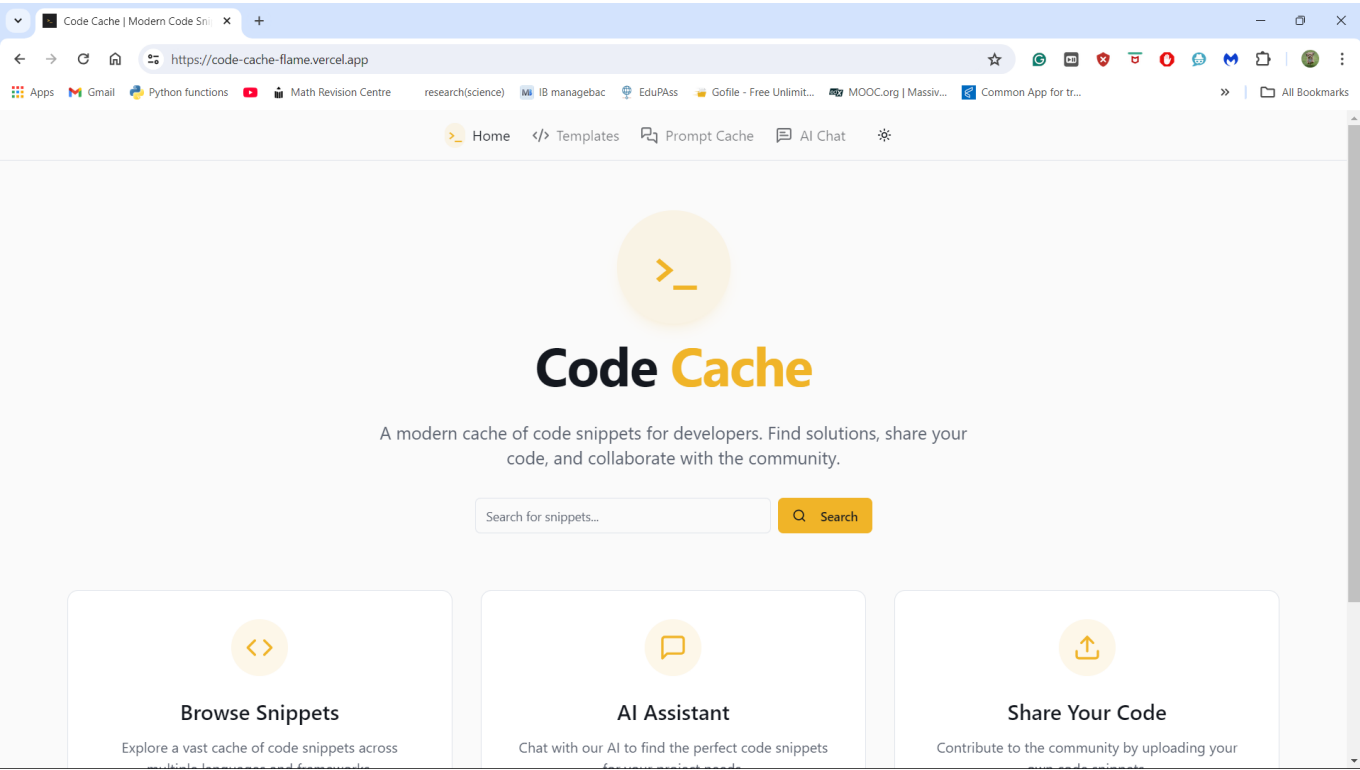
- Used to analyze code content and generate intelligent metadata such as tags and bio summaries based on semantic understanding.
- **Supabase Real-Time Database**
 - Code snippets and prompts are stored in a **PostgreSQL** database with real-time capabilities, synced to each authenticated user.
- **Search and Filtering**
 - Snippets can be filtered by **language**, **tags**, or searched by **title** to allow quick access and better content management.
- **Responsive Frontend with React + Styled-Components**
 - Built using React with modular styling via styled-components, ensuring consistent and responsive UI/UX across devices.
- **Routing with React Router**
 - Clean navigation between pages (login, dashboard, snippet editor, etc.) without reloading the app, improving performance and flow.
- **Deployment via Vercel with GitHub CI/CD**
 - The app is deployed on **Vercel**, automatically updated through GitHub commits using CI/CD pipelines for rapid iteration and delivery.

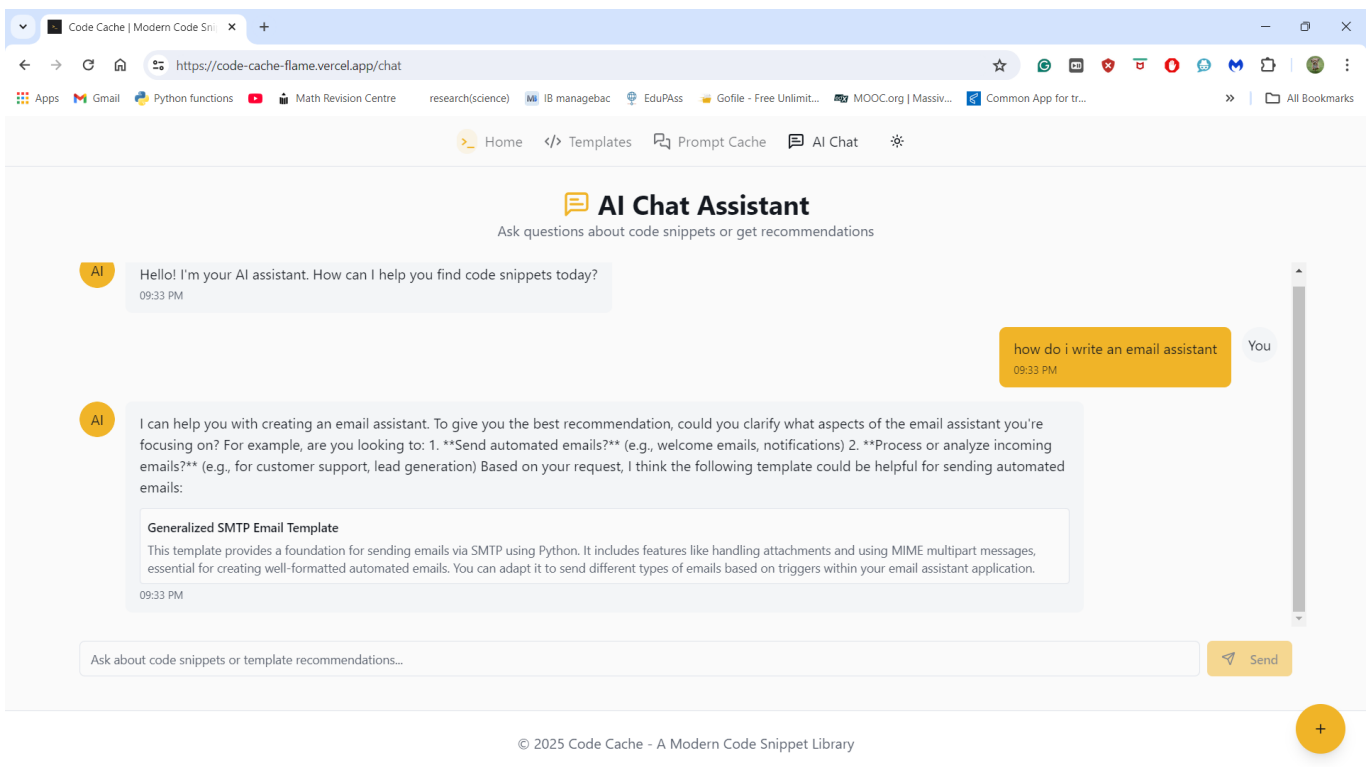
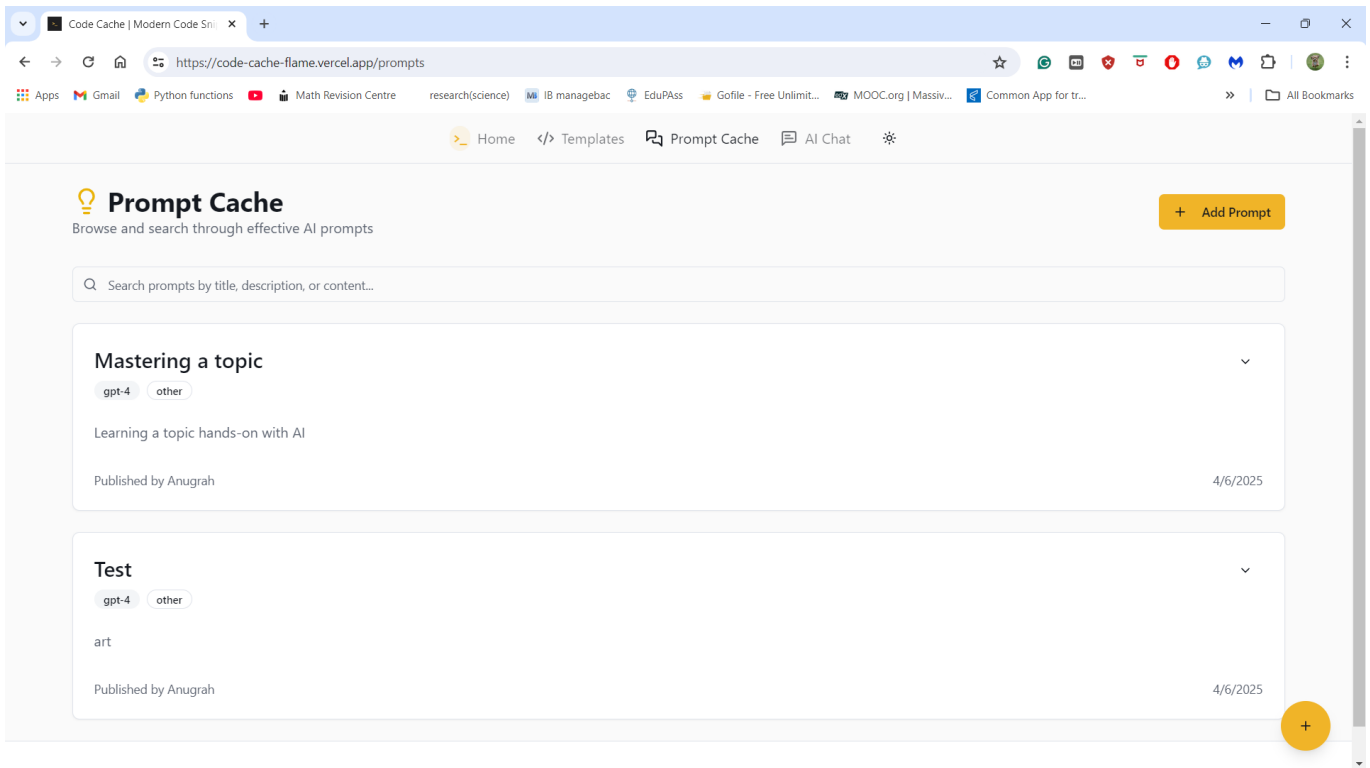
Code Snippets

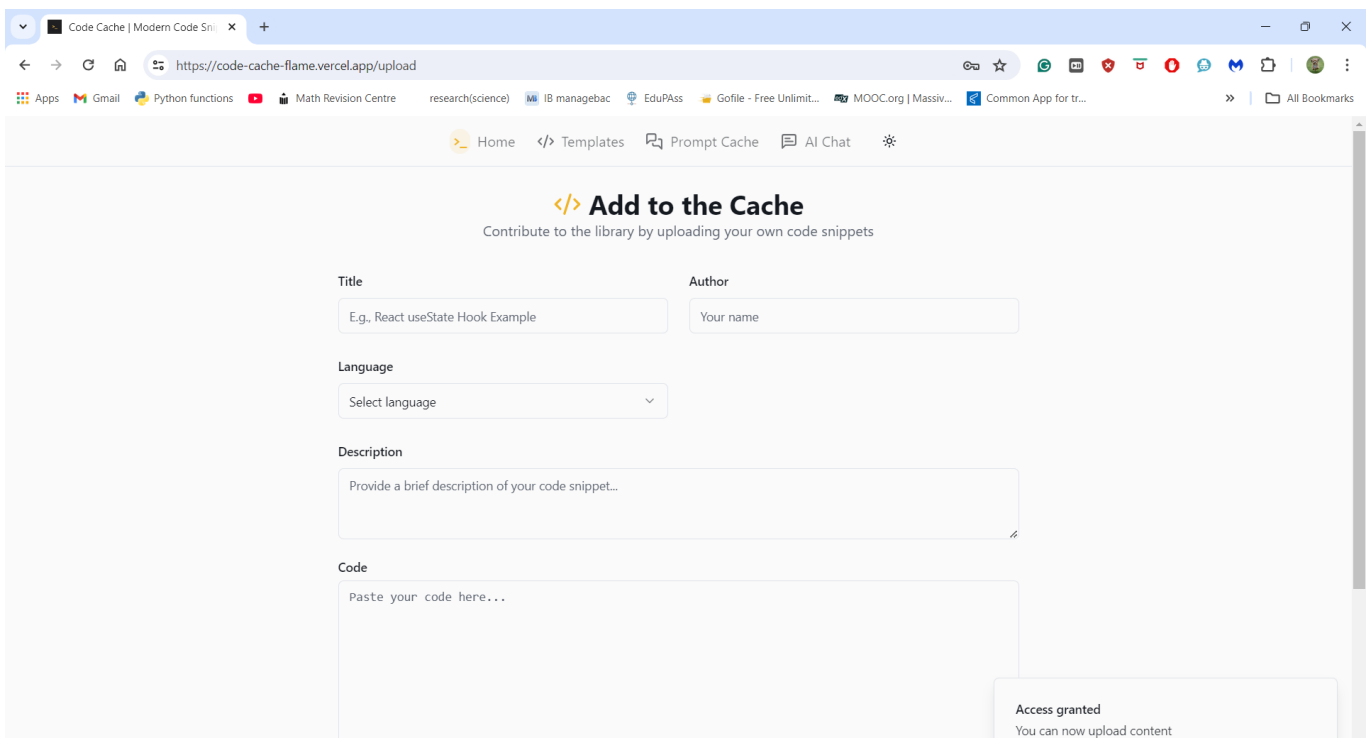
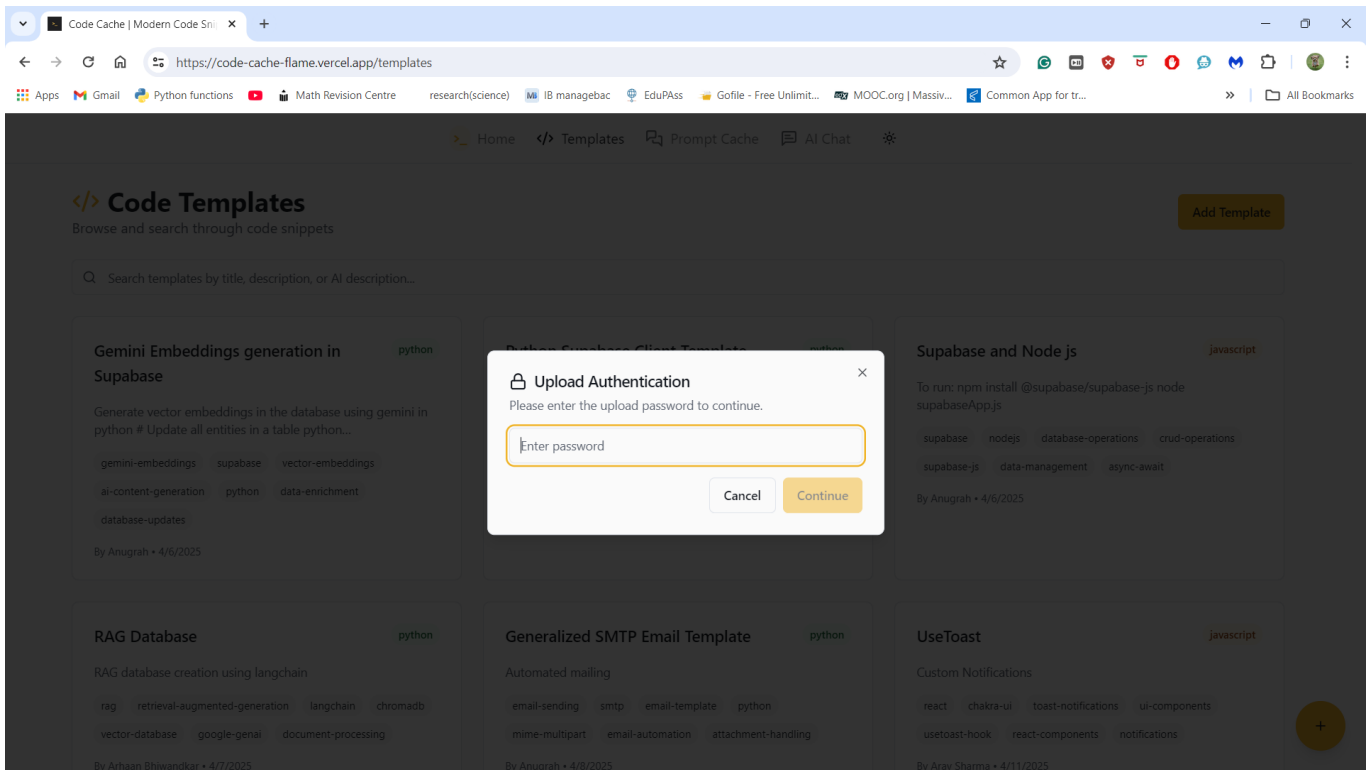
Code : <https://rb.gy/rpwwcl>

Implemented Website: <https://rb.gy/yuwh9u>

Output







Conclusion and Future Scope

Through CodeCache, we built a fully functional web application that lets users securely save, edit, and manage their personal code snippets in one place. Using React and Tailwind CSS for the frontend and Auth by Supabase for authentication and real-time data storage, we created a clean, responsive interface that works smoothly across devices. Every feature we implemented—from adding templates to database using password protection to snippet management—serves a clear purpose for everyday coding workflows. This project doesn't just work; it solves a real problem. CodeCache has the potential to become a go-to tool for developers who want quick access to their most-used code, anytime and anywhere.

While CodeCache currently supports core functionality, there is significant potential for future enhancements. Planned improvements include adding syntax highlighting for various languages, implementing a tagging system for better snippet organization, and introducing collaborative features that allow users to share and edit code together in real-time. Other ideas include version control for snippets, dark/light theme toggles, and code export options (PDF, TXT). Integration with platforms like GitHub or VS Code extensions could also boost productivity. These future upgrades aim to transform CodeCache into a more robust, community-driven developer tool.

Applications

1. Personal Code Library

Developers often write small, reusable pieces of code that they use across multiple projects. CodeCache serves as a personal library where users can store these snippets in an organized manner, saving time and avoiding the hassle of rewriting or digging through old projects.

2. Learning Resource for Students

For students learning programming, CodeCache can act as a study tool. They can store syntax references, sample programs, or important concepts in code form.

3. Bug Fix & Debug Archive

Developers frequently face recurring bugs or edge cases. CodeCache allows users to save solutions or workaround code, so when a similar issue arises, they have a go-to reference that speeds up the debugging process.

4. **Template Storage**

Whether it's HTML boilerplates, API call structures, CSS resets, or reusable components, templates are a staple in development. CodeCache makes it easy to save and access these templates for consistent and efficient development.

5. **Multi-Device Code Access**

Since CodeCache is cloud-based using Supabase, users can log in using only the database password and access their snippets from any device, at any time. This is ideal for developers who work across multiple machines or on-the-go.

6. **Quick Prototyping Support**

When building projects quickly—like during hackathons or MVP development—developers can pull in ready-made functions or logic from their snippet library in CodeCache, accelerating development without sacrificing code quality.

7. **Interview Preparation Tool**

Users can save frequently used data structures, algorithms, or problem-solving patterns to prepare for technical interviews. It becomes a handy, organized reference while practicing or revising before interviews.

8. **Organized Code Categorization**

CodeCache allows categorization of snippets by language or purpose, making it easier for users to manage their collection. Instead of searching through disorganized files or documents, everything is cleanly structured within the app.